Escola de Artes, Ciências e Humanidades — Universidade de São Paulo

RELATÓRIO DE CONFIGURAÇÃO DO AMBIENTE

Ambiente de Testes de Injeção de Falhas em Dispositivos IoT

Área: Sistemas de Informação / Segurança da Informação
Estudante: Eduardo Diniz da Silva
Orientador: Prof. Fábio Nakano

1 Resumo

Este relatório descreve a configuração do ambiente de testes de injeção de falhas em dispositivos IoT.

2 Configuração Inicial do Ambiente

A primeira etapa foi a instalação do QEMU, emulador de hardware, do GDB, depurador de código, do Newlib, biblioteca C padrão e do Make, ferramenta de automação de compilação.

```
sudo dnf install qemu-system-arm
sudo dnf install gdb
sudo dnf install arm-none-eabi-gcc arm-none-eabi-newlib
sudo dnf install make
```

Em seguida, foi feito o mapeamento físico da placa virtual no arquivo `linker.ld`, que define a memória FLASH e RAM, bem como o endereço do topo da pilha (stack) da RAM. Após isso, foi implementado o arquivo `startup.c`, que contém o código de inicialização do sistema. Ambos tiveram como referência o repositório (CORTEXM, 2026) e estão disponíveis no repositório do projeto https://github.com/D1n7z/ic-ascon. Além disso, foi criado o arquivo `Makefile`, que automatiza o processo de compilação do projeto:

- `make build`: compila o projeto e gera o binário`firmware.elf`.
- `make run`: executa o binário no QEMU
- `make clean`: remove os arquivos gerados durante a compilação para uma reconstrução limpa.

Finalizando a configuração inicial do ambiente, foi feito um teste de execução do binário no QEMU, através de um arquivo `main.c`

```
#include <stdio.h>

extern void initialise_monitor_handles(void);

int main(void) {
    initialise_monitor_handles();

```

```
8           printf("\nHello, World!\n");
9           return 0;
10      }
```

que imprimiu a mensagem "Hello, World!"no console do terminal. Com isso, a parte inicial de instalação e configuração do QEMU foi concluída com sucesso.

A próxima etapa foi a configuração do GDB, que é o depurador de código. Para isso, foi criado um novo `main.c` com um código simples de soma para testar o GDB:

```
1       #include <stdio.h>
2
3       extern void initialise_monitor_handles(void);
4
5       int main(void) {
6           initialise_monitor_handles();
7
8           volatile int soma = 0;
9
10          printf("Inicio da execucao do programa\n");
11          for (int i = 0; i < 5; i++) {
12              soma += i;
13          }
14          printf("Soma: %d\n", soma);
15
16          return 0;
17      }
```

Referências Bibliográficas

CORTEXM. **baremetal: ARM Cortex-M bare-metal example code in C++ and CMake build**. [S.l.]: GitHub, 2026. Disponível em: <https://github.com/cortexm/baremetal>.